# AKS and kubectl

## Create Your Cluster and Take Command




Daniel Colón

# Azure Kubernetes Service and kubectl

A tour of the AKS portal: Kubernetes resources and Settings, with the kubectl and az commands behind each one





Agenda

# 90 minutes, following the AKS portal menu

**10**

minutes

## Kubernetes and AKS basics

What AKS manages for you, and how kubectl talks to the cluster

**10**

minutes

## Demo 1: create a cluster

aks.ps1 builds a 2-node cluster and connects kubectl

**40**

minutes

## Kubernetes resources

Namespaces, workloads, services, storage, config and more, plus Demo 2

**20**

minutes

## Settings

Node pools, upgrades, scaling, networking, extensions, backup, mesh, GitOps, policy

**10**

minutes

## Wrap-up and Q&A

Clean up resources, recap and questions

# What Kubernetes does

Kubernetes runs containers across a group of machines and keeps them in the state you describe.

You declare what you want in YAML. Controllers compare that with what is running and fix the difference, continuously.

**Desired state → Observe → Act**
The reconcile loop behind every resource in this deck

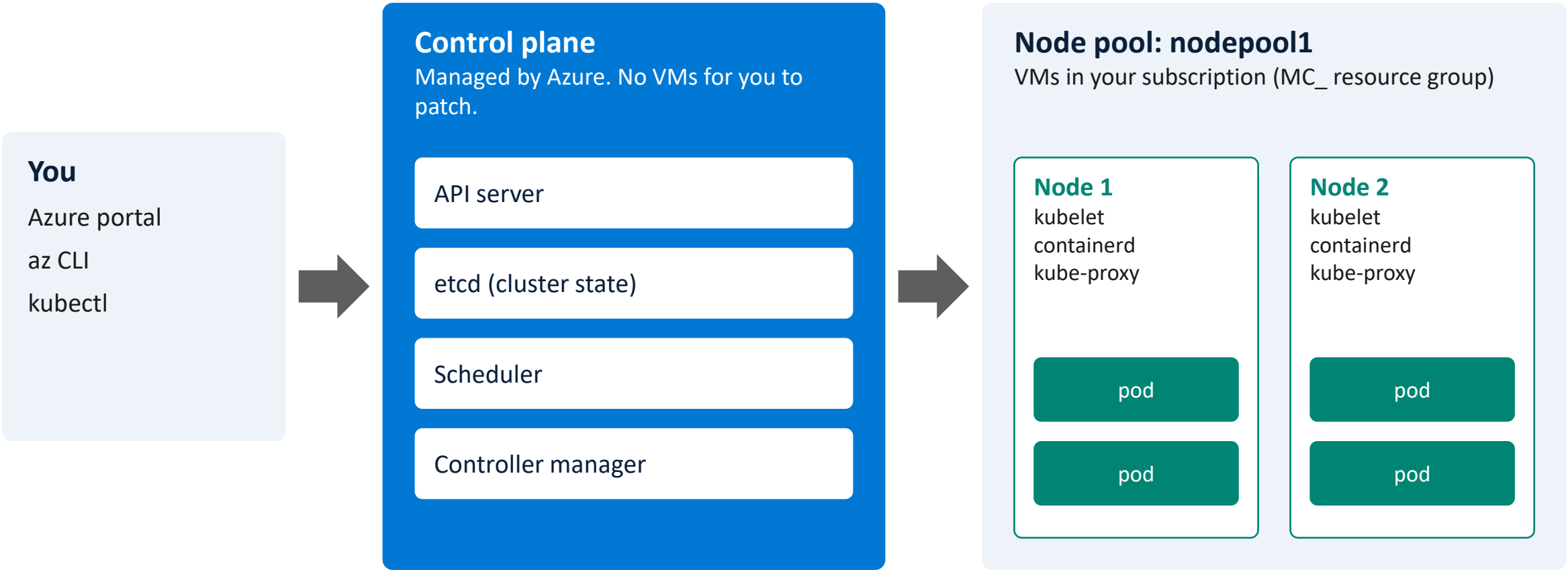
### Self-healing
Restarts failed containers and replaces pods when a node fails.

### Scaling
Add or remove replicas by hand or automatically, from pods up to nodes.

### Rolling updates
Ship new versions gradually and roll back in seconds.

### Service discovery
Stable names and IPs for groups of pods, with built-in load balancing.

# How AKS splits the work




*Free tier: no uptime SLA. Standard tier: financially backed API server SLA. You pay for the nodes either way.*

# Reading a kubectl command

```
kubectl get pods -n demo -o wide
```

| the CLI | verb | resource type | namespace | output format |
|---|---|---|---|---|

```
# Which cluster am I talking to?
kubectl config get-contexts
kubectl config use-context aks-1234
# Discover resource types and their fields
kubectl api-resources
kubectl explain deployment.spec
```

```
# Variables used on later slides
# (names are printed by aks.ps1)
$rg  = "rg-aks-1234"
$aks = "aks-1234"
# Verbs you will see most
get · describe · apply · delete
logs · exec · scale · rollout
```

LIVE DEMO 1

# Create an AKS cluster with aks.ps1

1. **Install the tools**
   az aks install-cli installs kubectl and kubelogin
2. **Create a resource group**
   rg-aks-<random> in eastus, tagged env=demo
3. **Create the cluster**
   2 × Standard_D4s_v3 nodes, managed identity (5–10 min)
4. **Connect kubectl**
   az aks get-credentials updates ~/.kube/config
5. **Verify**
   kubectl config current-context, kubectl cluster-info

```
# In PowerShell, from the demo folder
az login
.\aks.ps1

# Then
kubectl get nodes
kubectl get ns
```

**While it provisions:** open the cluster in the Azure portal and walk the left menu: Kubernetes resources and Settings.



Azure portal › Kubernetes service › your cluster

# The AKS menu this deck follows

## Kubernetes resources

**Namespaces**

**Workloads** Deployments · Pods · Replica sets · Stateful sets · Daemon sets · Jobs · Cron jobs

**Services and ingresses** Services · Ingresses

**Storage** Persistent volume claims · Persistent volumes · Storage classes

**Configuration** Config maps · Secrets

**Custom resources**

**Events**

**Run command**

*Helm* *not covered*

## Settings

**Node pools**

**Cluster configuration**

**Application scaling**

**Networking**

**Extensions + applications**

**Backup**

**Service mesh - Istio**

**GitOps**

**Automated deployments**

**Policies**

*Properties · Locks* *not covered*

# Kubernetes resources



The objects running inside the cluster, seen through the portal and kubectl

- **Namespaces**
- **Workloads**
- **Services and ingresses**
- **Storage**
- **Configuration**
- **Custom resources**
- **Events**
- **Run command**





# Namespaces

*Virtual clusters inside one physical cluster: a scope for names, access and quotas*

### Isolation boundary

Names are unique per namespace. RBAC, resource quotas and network policies apply per namespace.

### Built-in namespaces

default, kube-system, kube-public and kube-node-lease. Add-ons create their own, e.g. gatekeeper-system.

### Cluster-scoped objects

Nodes, persistent volumes, storage classes and CRDs belong to no namespace.

### In the portal

Lists every namespace with status and age. + Create › Namespace, or apply YAML. Other blades filter by namespace.

```
# List namespaces (ns is the short name)
kubectl get ns
# Create a namespace and list everything in it
kubectl create namespace demo
kubectl get all -n demo
# Make demo the default namespace for the current context
kubectl config set-context --current --namespace=demo
# Delete a namespace and everything inside it
kubectl delete namespace demo
```

LIVE DEMO 2

# Deploy hello-nginx.yaml




```
kubectl apply -f hello-nginx.yaml
kubectl get all -n demo
kubectl get svc hello-nginx -n demo --watch
# Browse to http://<EXTERNAL-IP>
```

**What to show**

The page shows which pod served it. Refresh, scale to 4, delete a pod, and find each object in the portal blades.

*Walkthrough: demo/hello-nginx.md*



Kubernetes resources › Workloads › Deployments

# Deployments

*Declare the desired state of a stateless app, and Kubernetes keeps it that way*

### Desired state

Image, replica count, labels and pod template. The controller reconciles toward it continuously.

### Rolling updates

Changing the pod template creates a new ReplicaSet and swaps pods gradually, with zero downtime by default.

### Rollback

Each change is a revision. Undo returns to the previous ReplicaSet in seconds.

### In the portal

Workloads › Deployments: ready count, namespace and age. Open one for YAML, events and its pods.

```
# List and inspect deployments
kubectl get deploy -n demo
kubectl describe deploy hello-nginx -n demo
# Scale out
kubectl scale deploy hello-nginx --replicas=4 -n demo
# Rolling update to a new image, then watch it
kubectl set image deploy/hello-nginx nginx=nginx:1.28 -n demo
kubectl rollout status deploy/hello-nginx -n demo
# Roll back to the previous revision
kubectl rollout undo deploy/hello-nginx -n demo
```





# Pods

*The smallest deployable unit: one or more containers that share an IP address and storage*

### Ephemeral

Pods are replaced, not repaired. A new pod gets a new name and IP, so use Services to reach them.

### Lifecycle

Pending → Running → Succeeded or Failed. Watch for CrashLoopBackOff and ImagePullBackOff.

### Probes & resources

Liveness and readiness probes report health. CPU and memory requests guide scheduling.

### In the portal

Workloads › Pods: status, node, pod IP and restarts. Open a pod for YAML, events and container logs.

```
# List pods with their node and IP
kubectl get pods -n demo -o wide
# Store the first pod's name (pod/<name>)
$pod = kubectl get pods -n demo -o name | Select-Object -First 1
# Inspect, read logs, open a shell
kubectl describe $pod -n demo
kubectl logs $pod -n demo --tail=20
kubectl exec -it $pod -n demo -- sh
# Forward local port 8080 to the pod's port 80
kubectl port-forward $pod 8080:80 -n demo
```





# Replica sets

*Keeps a stable number of identical pods running, and is usually managed for you by a Deployment*

### Desired count

Watches pods that match its label selector and creates or deletes pods to match replicas.

### Owned by Deployments

Each Deployment revision gets its own ReplicaSet with a hash suffix. Rarely created directly.

### Revision history

Old ReplicaSets are scaled to 0 and kept (revisionHistoryLimit: 10), so rollbacks are instant.

### In the portal

Workloads › Replica sets: desired vs. ready pods per ReplicaSet. Old revisions show 0/0.

```
# One ReplicaSet per Deployment revision
kubectl get rs -n demo
kubectl describe rs -n demo -l app=hello-nginx
# Self-healing: delete a pod and the ReplicaSet replaces it
kubectl delete $pod -n demo
kubectl get pods -n demo --watch
# The revisions behind the Deployment
kubectl rollout history deploy/hello-nginx -n demo
```





# Stateful sets

*For stateful apps that need stable names, stable storage and an ordered rollout*

### Stable identity

Pods are named app-0, app-1, … and keep their name and DNS entry through a headless Service.

### Storage per pod

volumeClaimTemplates create one PVC per pod. The PVC follows the pod when it is rescheduled.

### Ordered operations

Pods start, scale and update one at a time, in order. Used for databases, Kafka and ZooKeeper.

### In the portal

Workloads › Stateful sets: ready replicas per StatefulSet. Check Storage for the claims it created.

```
# StatefulSets in all namespaces
kubectl get sts -A
# Inspect one and its pods (ordinal names: <name>-0, <name>-1)
kubectl describe sts <name> -n <namespace>
kubectl get pods -n <namespace> -l app=<label>
# Scale: pods are added or removed in order
kubectl scale sts <name> --replicas=3 -n <namespace>
# One PersistentVolumeClaim per pod
kubectl get pvc -n <namespace>
```





# Daemon sets

*Runs one copy of a pod on every node, or on every node that matches*

### Node agents

Ideal for log collectors, monitoring agents, and network and storage drivers.

### Follows the nodes

When a node joins, the pod is added automatically. When a node leaves, its pod is removed.

### AKS uses them

kube-proxy, the Azure Disk and Azure Files CSI drivers and monitoring agents run as DaemonSets.

### In the portal

Workloads › Daemon sets: desired vs. ready per DaemonSet. Filter to kube-system to see AKS's own.

```
# DaemonSets across the cluster
kubectl get ds -A
# AKS system DaemonSets
kubectl get ds -n kube-system
kubectl describe ds kube-proxy -n kube-system
# One pod per node: compare the NODE column
kubectl get pods -n kube-system -o wide | Select-String kube-proxy
```





# Jobs

*Run a task to completion, then stop*

### Run to completion

Pods run until they exit successfully. Good for migrations, batch processing and one-off scripts.

### Retries & parallelism

backoffLimit sets retries. completions and parallelism control how many pods run and succeed.

### Clean-up

Finished Jobs stay so you can read logs. ttlSecondsAfterFinished deletes them automatically.

### In the portal

Workloads › Jobs: completions (e.g. 1/1), duration and age. Open a Job to see its pods and logs.

```
# Create a one-off Job
kubectl create job hello --image=busybox:1.36 -n demo -- echo "Hello from a Job"
# Watch it complete
kubectl get jobs -n demo
kubectl wait --for=condition=complete job/hello -n demo --timeout=60s
# Read its output, then delete it
kubectl logs job/hello -n demo
kubectl delete job hello -n demo
```





# Cron jobs

*Create Jobs on a schedule, using cron syntax*

### Schedule

Standard cron format: "0 2 * * *" runs daily at 02:00. AKS uses UTC unless timeZone is set.

### Concurrency

concurrencyPolicy (Allow, Forbid, Replace) decides what happens if the last run is still going.

### History & suspend

Keeps the last 3 successful and 1 failed Job by default. suspend: true pauses the schedule.

### In the portal

Workloads › Cron jobs: schedule, suspend state and last schedule time.

```
# Run every minute
kubectl create cronjob hello-cron --image=busybox:1.36 --schedule="*/1 * * * *" -n demo -- date
# See the schedule and the Jobs it creates
kubectl get cronjob -n demo
kubectl get jobs -n demo --watch
# Trigger a run now, outside the schedule
kubectl create job hello-manual --from=cronjob/hello-cron -n demo
# Remove it
kubectl delete cronjob hello-cron -n demo
```





# Services

*A stable IP address and DNS name in front of a changing set of pods*

### Label selector

Routes to every ready pod whose labels match. EndpointSlices track the current pod IPs.

### Types

ClusterIP (internal, default), NodePort, LoadBalancer (Azure Load Balancer and public IP), ExternalName.

### Internal DNS

Reach it as hello-nginx in the same namespace, or hello-nginx.demo.svc.cluster.local.

### In the portal

Services and ingresses › Services: type, cluster IP, external IP and ports. Click the external IP to browse.

```
# Services and their external IPs
kubectl get svc -n demo
kubectl describe svc hello-nginx -n demo
# Pod IPs behind the Service
kubectl get endpointslices -n demo
# Just the public IP
kubectl get svc hello-nginx -n demo -o jsonpath="{.status.loadBalancer.ingress[0].ip}"
# Add an internal-only (ClusterIP) Service for the same pods
kubectl expose deploy hello-nginx --name=hello-internal --port=80 -n demo
```

# Ingresses

*HTTP(S) routing: many apps behind one IP address, by host name and path*

### Layer 7 rules

Route by host (shop.contoso.com) and path (/api) to Services. Terminate TLS with a Secret.

### Needs a controller

AKS options: the application routing add-on (managed NGINX) or Application Gateway for Containers.

### What's next

Gateway API is the successor to Ingress, with richer routing. Consider it for new designs.

### In the portal

Services and ingresses › Ingresses: class, hosts, address and ports for each Ingress.

```
# Enable the managed NGINX ingress controller (application routing add-on)
az aks approuting enable -g $rg -n $aks
kubectl get ingressclass
# Route all paths to the hello-nginx Service
kubectl create ingress hello --class=webapprouting.kubernetes.azure.com --rule="/*=hello-nginx:80" -n demo
# Find the ingress address
kubectl get ingress -n demo
kubectl describe ingress hello -n demo
```





# Persistent volume claims

*An app's request for storage: size, access mode and storage class*

### Request, not disk

The pod refers to the claim. Kubernetes finds or creates a matching PersistentVolume and binds them.

### Access modes

ReadWriteOnce: one node (Azure Disk). ReadWriteMany: many nodes (Azure Files).

### Dynamic provisioning

Name a storage class and AKS creates the Azure disk or file share for you.

### In the portal

Storage › Persistent volume claims: status, bound volume, capacity, access mode and storage class.

```
# Create a 5 GiB Azure Disk claim
@"
apiVersion: v1
kind: PersistentVolumeClaim
metadata: { name: data }
spec: { accessModes: [ReadWriteOnce], storageClassName: managed-csi, resources: { requests: { storage: 5Gi } } }
"@ | kubectl apply -n demo -f -
# Pending until a pod uses it (WaitForFirstConsumer); then delete it
kubectl describe pvc data -n demo
kubectl delete pvc data -n demo
```





# Persistent volumes

*The actual piece of storage: an Azure disk, file share or blob container*

### Cluster-scoped

PVs belong to no namespace. Each one binds to exactly one claim.

### Reclaim policy

Delete (dynamic default) removes the Azure resource with the claim. Retain keeps it for recovery.

### Where it lives

Dynamically created disks live in the node resource group (MC_…) next to the cluster's VMs.

### In the portal

Storage › Persistent volumes: capacity, access modes, reclaim policy, status and the bound claim.

```
# Volumes, status and the claims they serve
kubectl get pv
kubectl describe pv <volume>
# Name, claim and reclaim policy in one view
kubectl get pv -o custom-
columns=NAME:.metadata.name,CLAIM:.spec.claimRef.name,POLICY:.spec.persistentVolumeReclaimPolicy
# Find the Azure disks behind them
$nodeRg = az aks show -g $rg -n $aks --query nodeResourceGroup -o tsv
az disk list -g $nodeRg -o table
```





# Storage classes

*Templates that describe how storage is created on demand*

### Built into AKS

default and managed-csi (Standard SSD), managed-csi-premium, azurefile-csi, azurefile-csi-premium.

### Key settings

Provisioner (CSI driver), SKU parameters, reclaimPolicy, allowVolumeExpansion and volumeBindingMode.

### The default class

Marked (default) and used when a claim names no class. Create your own for other SKUs.

### In the portal

Storage › Storage classes: provisioner, reclaim policy and binding mode for each class.

```
# Classes available in the cluster
kubectl get storageclass
kubectl describe storageclass managed-csi-premium
# Full definition, including parameters
kubectl get storageclass default -o yaml
# CSI drivers that implement them
kubectl get csidrivers
```

# Config maps

*Non-sensitive configuration kept outside the container image*

### Key/value or files

Store settings or whole files such as nginx.conf or index.html. hello-nginx uses both.

### Two ways to use

As environment variables, or mounted as files. Mounted files update in place; env vars need a restart.

### Not for secrets

Values are plain text, readable by anyone with access to the namespace. Max size 1 MiB.

### In the portal

Configuration › Config maps: keys and age. Open one to view its data as YAML.

```
# Config maps in the demo namespace
kubectl get cm -n demo
kubectl describe cm hello-html -n demo
# Full content as YAML
kubectl get cm nginx-conf -n demo -o yaml
# Create from literals or from a file
kubectl create configmap app-settings --from-literal=COLOR=blue --from-literal=MODE=demo -n demo
kubectl create configmap site --from-file=index.html -n demo
# After changing a config map, restart the pods to pick it up
kubectl rollout restart deploy/hello-nginx -n demo
```





# Secrets

*Sensitive values such as passwords, keys and certificates*

### Encoded, not encrypted

Values are base64 in the API. AKS encrypts etcd at rest; add KMS with Azure Key Vault for your own keys.

### Types

Opaque (generic), kubernetes.io/tls for certificates, kubernetes.io/dockerconfigjson for registries.

### Better: Key Vault

The Secrets Store CSI Driver add-on mounts secrets directly from Azure Key Vault.

### In the portal

Configuration › Secrets: type and key count per secret. Treat access to this blade as privileged.

```
# List secrets
kubectl get secrets -n demo
# Create a generic secret (password from an environment variable)
kubectl create secret generic db-cred --from-literal=username=app --from-literal=password=$env:DB_PASSWORD -n demo
kubectl describe secret db-cred -n demo
# Read and decode one value
$b64 = kubectl get secret db-cred -n demo -o jsonpath="{.data.username}"
[Text.Encoding]::UTF8.GetString([Convert]::FromBase64String($b64))
# TLS secret for an ingress
kubectl create secret tls hello-tls --cert=tls.crt --key=tls.key -n demo
```

# Custom resources

*Extend the Kubernetes API with your own object types*

### CRDs

A CustomResourceDefinition adds a new object type, such as ScaledObject, to the Kubernetes API.

### Operators

A controller watches the custom objects and does the work: certificates, databases, scaling.

### On AKS

Add-ons bring their own: KEDA, Flux, Istio and Azure Policy (Gatekeeper).

### In the portal

Custom resources: lists the CRDs in the cluster. Open one to browse its objects.

```
# Custom resource definitions installed in the cluster
kubectl get crd
# All resource types, built-in and custom
kubectl api-resources
# The schema of a kind
kubectl explain <kind>.spec
# Objects of a custom kind
kubectl get <plural-name> -A
```

# Events

*The cluster's activity log, and the first place to look when something goes wrong*

### What gets recorded

Scheduling, image pulls, container starts, probe failures, scaling and volume attach operations.

### Normal vs. Warning

Filter on Warning to find problems fast: FailedScheduling, BackOff, FailedMount, Unhealthy.

### Short-lived

Kept for about an hour by default. Send them to Container insights to keep history.

### In the portal

Events: type, reason, object and message across the cluster, filterable by namespace.

```
# Recent events, newest last
kubectl get events -n demo --sort-by=.lastTimestamp
# Only warnings, cluster-wide
kubectl get events -A --field-selector type=Warning
# Events for a single object
kubectl events --for $pod -n demo
# Stream new events as they happen
kubectl get events -n demo --watch
```

# Run command

*Run kubectl and helm against the cluster without direct network access*

### How it works

AKS starts a short-lived pod in the aks-command namespace, runs your command and returns the output.

### When to use it

Private clusters, locked-down API servers, or a quick check from the portal with no local tools.

### Limits

Not interactive: no exec -it and no --watch. Uses your Azure identity and the cluster's RBAC.

### In the portal

Run command: a terminal-like pane in the portal. Type a command and attach files such as manifests.

```
# Run a command through the AKS API
az aks command invoke -g $rg -n $aks --command "kubectl get pods -n demo"
# Send a local file along with the command
az aks command invoke -g $rg -n $aks --command "kubectl apply -f hello-nginx.yaml" --file hello-nginx.yaml
# Get the result of an earlier run
az aks command result -g $rg -n $aks --command-id <command-id>
```

# Settings



Configure the AKS resource itself: mostly az aks, ARM and add-ons

- **Node pools**
- **Cluster configuration**
- **Application scaling**
- **Networking**
- **Extensions + applications**
- **Backup**
- **Service mesh - Istio**
- **GitOps**
- **Automated deployments**
- **Policies**



Settings › Node pools

# Node pools

*Groups of identical VMs (virtual machine scale sets) that run your pods*

### System vs. user

System pools host critical add-ons such as CoreDNS. Apps belong on user pools. aks.ps1 created nodepool1.

### Scale

Set a node count, or use the cluster autoscaler with a min and max. Spot pools cut cost.

### Placement

Labels, taints and node selectors steer pods to the right pool: GPU, Windows or larger VMs.

### In the portal

Node pools: mode, VM size, node count and version per pool. The Nodes tab lists every VM.

```
# Node pools and their nodes
az aks nodepool list -g $rg --cluster-name $aks -o table
kubectl get nodes -L agentpool,kubernetes.azure.com/mode
# Add a user pool, then scale it
az aks nodepool add -g $rg --cluster-name $aks -n userpool --mode User --node-count 1 --node-vm-size
Standard_D4s_v3
az aks nodepool scale -g $rg --cluster-name $aks -n userpool --node-count 2
# Turn on the cluster autoscaler
az aks nodepool update -g $rg --cluster-name $aks -n userpool --enable-cluster-autoscaler --min-count 1 --
max-count 3
```



Settings › Cluster configuration

# Cluster configuration

*Kubernetes version, upgrades, and how users sign in and are authorized*

### Version & upgrades

Upgrade the control plane, then node pools. Auto-upgrade channels: patch, stable, rapid, node-image.

### Node OS updates

The node OS upgrade channel (NodeImage, SecurityPatch) applies OS patches in a maintenance window.

### Authentication

Local accounts with Kubernetes RBAC (aks.ps1), Entra ID with Kubernetes RBAC, or Entra ID with Azure RBAC.

### In the portal

Cluster configuration: Kubernetes version, upgrade settings and the authentication and authorization options.

```
# Current version and available upgrades
az aks get-upgrades -g $rg -n $aks -o table
# Upgrade the control plane and all node pools
az aks upgrade -g $rg -n $aks --kubernetes-version <version>
# Stay current automatically
az aks update -g $rg -n $aks --auto-upgrade-channel patch --node-os-upgrade-channel NodeImage
# Version and auth settings
az aks show -g $rg -n $aks --query "{version:kubernetesVersion, entraId:aadProfile,
localAccountsDisabled:disableLocalAccounts}"
kubectl version
```



Settings › Application scaling

# Application scaling

*Scale pods automatically on CPU, memory or external events*

### Horizontal Pod Autoscaler

Adds or removes replicas based on CPU or memory use. Pods need resource requests.

### KEDA add-on

Event-driven scaling from many sources: Service Bus, Kafka, Prometheus, cron. Can scale to zero.

### Works with nodes

More pods than fit? The cluster autoscaler on the node pool adds nodes to make room.

### In the portal

Application scaling: turn the KEDA add-on on or off for the cluster.

```
# Enable the KEDA add-on
az aks update -g $rg -n $aks --enable-keda
kubectl get pods -n kube-system | Select-String keda
# CPU-based autoscaling for a deployment (needs CPU requests)
kubectl autoscale deploy hello-nginx --cpu-percent=50 --min=2 --max=5 -n demo
kubectl get hpa -n demo
# KEDA scalers you have defined
kubectl get scaledobjects -A
```



Settings › Networking

# Networking

*How pods get IP addresses, how traffic flows, and who can reach the API server*

### Network plugin

Azure CNI Overlay, Azure CNI with a pod subnet, or Azure CNI powered by Cilium. Kubenet is retiring.

### Network policy

Control pod-to-pod traffic with Azure, Calico or Cilium network policies.

### API server access

Public with authorized IP ranges, or a private cluster. Outbound via load balancer or NAT gateway.

### In the portal

Networking: network plugin, pod and service CIDRs, DNS, load balancer and API server access.

```
# Network profile of the cluster
az aks show -g $rg -n $aks --query networkProfile
# Pod IPs and network policies
kubectl get pods -A -o wide
kubectl get networkpolicy -A
# Test Service DNS from inside the cluster
kubectl run tmp --rm -it --image=busybox:1.36 -n demo -- wget -qO- http://hello-nginx
# Allow API server access only from your IP
az aks update -g $rg -n $aks --api-server-authorized-ip-ranges <your-public-ip>/32
```



Settings › Extensions + applications

# Extensions + applications

*Install and manage supported add-on software through Azure Resource Manager*

### Cluster extensions

Microsoft and partner software installed and upgraded by Azure: Flux, Dapr, Azure Machine Learning, Backup.

### Kubernetes apps

Azure Marketplace offers packaged as extensions and billed through your Azure subscription.

### Managed lifecycle

Version, auto-upgrade and configuration settings are tracked in Azure, not only in the cluster.

### In the portal

Extensions + applications: installed extensions with version and state. + Add browses the catalog.

```
# One-time: install the CLI extension
az extension add --name k8s-extension
# Installed extensions
az k8s-extension list -g $rg -c $aks -t managedClusters -o table
# Install Dapr as an example
az k8s-extension create -g $rg -c $aks -t managedClusters -n dapr --extension-type Microsoft.Dapr
kubectl get pods -n dapr-system
# Remove it
az k8s-extension delete -g $rg -c $aks -t managedClusters -n dapr --yes
```



Settings › Backup

# Backup

*Protect cluster state and persistent volumes with Azure Backup*

### What's backed up

Kubernetes resources (deployments, config maps, secrets…) and Azure Disk volumes as snapshots.

### Building blocks

The backup extension, a storage account blob container, a Backup vault and Trusted Access.

### Restore options

Restore all or selected namespaces to the same cluster or another one.

### In the portal

Backup: a Configure backup wizard that installs the extension and links a Backup vault.

```
# Backup extension status
az k8s-extension show -g $rg -c $aks -t managedClusters -n azure-aks-backup
kubectl get pods -n dataprotection-microsoft
# Trusted Access between the vault and the cluster
az aks trustedaccess rolebinding list -g $rg --cluster-name $aks -o table
# Backup instances protected by a vault
az dataprotection backup-instance list -g $rg --vault-name <vault> -o table
```



Settings › Service mesh - Istio

# Service mesh - Istio

*An Istio-based service mesh delivered as a managed AKS add-on*

### Security

Automatic mutual TLS between services, plus fine-grained authorization policies.

### Traffic control

Canary and blue/green releases, retries, timeouts and fault injection without code changes.

### Managed revisions

Microsoft supplies tested Istio revisions (asm-1-xx). Label a namespace to inject sidecars.

### In the portal

Service mesh - Istio: enable the add-on, pick the revision and add ingress gateways.

```
# Supported Istio revisions in your region
az aks mesh get-revisions -l eastus -o table
# Enable the add-on and check the control plane
az aks mesh enable -g $rg -n $aks
kubectl get pods -n aks-istio-system
# Inject sidecars into the demo namespace (use your revision)
kubectl label namespace demo istio.io/rev=asm-1-xx
kubectl rollout restart deploy/hello-nginx -n demo
# Add an external ingress gateway
az aks mesh enable-ingress-gateway -g $rg -n $aks --ingress-gateway-type external
```



Settings › GitOps

# GitOps

*Git is the source of truth, and Flux keeps the cluster in sync with it*

### Pull-based

Flux v2, a cluster extension, watches a Git repo, OCI artifact or bucket and applies what it finds.

### Drift correction

Manual changes are reverted on the next reconcile. Every change is a reviewed commit.

### At scale

Apply the same configuration to many clusters, and enforce it with Azure Policy.

### In the portal

GitOps: create a Flux configuration (source, branch, kustomizations) and watch its compliance state.

```
# One-time: install the CLI extension
az extension add --name k8s-configuration
# Point Flux at a repository (installs the Flux extension if needed)
az k8s-configuration flux create -g $rg -c $aks -t managedClusters -n app-config --namespace app-config --
scope cluster -u <git-repo-url> --branch main --kustomization name=apps path=./apps prune=true
# Configuration and compliance state
az k8s-configuration flux list -g $rg -c $aks -t managedClusters -o table
# Flux objects inside the cluster
kubectl get gitrepositories,kustomizations -A
```



Settings › Automated deployments

# Automated deployments

*Generate a CI/CD pipeline from your repository to AKS in a few clicks*

### What it creates

A GitHub Actions workflow, plus a Dockerfile and manifests if needed, as a pull request.

### The pipeline

Build the image → push to Azure Container Registry → deploy with manifests, Helm or Kustomize.

### No stored passwords

Uses OpenID Connect with a federated identity instead of secrets stored in GitHub.

### In the portal

Automated deployments: + Create, connect GitHub, choose repo, registry and namespace, then merge the PR.

```
# Let the cluster pull from your registry
az aks update -g $rg -n $aks --attach-acr <acr-name>
az acr repository list -n <acr-name> -o table
# Watch the generated workflow run (GitHub CLI)
gh run list --workflow <workflow-file>.yml
gh run watch
# Confirm the rollout in the cluster
kubectl rollout status deploy/<app> -n <namespace>
```



Settings › Policies

# Policies

*Enforce and audit rules for what may run in the cluster, using Azure Policy*

### How it works

The Azure Policy add-on runs Gatekeeper (OPA) as an admission controller and reports compliance to Azure.

### Built-in initiatives

Pod security baseline and restricted, allowed registries, no privileged containers, require limits.

### Audit or deny

Start with Audit to see violations, then switch to Deny to block non-compliant deployments.

### In the portal

Policies: add-on status and compliance per policy or initiative, with the non-compliant resources.

```
# Enable the Azure Policy add-on
az aks enable-addons -g $rg -n $aks --addons azure-policy
kubectl get pods -n gatekeeper-system
kubectl get pods -n kube-system | Select-String azure-policy
# Assigned policies arrive as Gatekeeper constraints
kubectl get constrainttemplates
kubectl get constraints
```



Wrap-up

# Clean up so you stop paying

### Remove only the sample app

The cluster keeps running and the Azure public IP for the Service is released.

```
kubectl delete -f hello-nginx.yaml
# or
kubectl delete namespace demo
```

### Delete everything

Deletes the resource group, the cluster and the MC_ node resource group with all VMs and disks.

```
az group delete --name $rg --yes --no-
wait
az group exists --name $rg
```



Deleting a resource group removes every resource in it. Check that $rg is the demo group before running the command. az group exists returns false when deletion is complete.

# Recap and Q&A

Everything in the portal is the Kubernetes API or the AKS resource, and kubectl or az can do it too

### Kubernetes resources

Read from the cluster's API: the same objects kubectl get shows.

### Settings

Change the Azure resource and its add-ons: az aks and az k8s-extension.

### Learn more

learn.microsoft.com/azure/aks

kubernetes.io/docs/reference/kubectl



# References

- Azure Kubernetes Service (AKS) documentation
  https://learn.microsoft.com/en-us/azure/aks/
- Azure Command-Line Interface (CLI) documentation
  https://learn.microsoft.com/en-us/cli/azure/?view=azure-cli-latest
- kubectl Command-Line Interface (CLI) documentation
  https://kubernetes.io/docs/reference/kubectl/